

EE 309 - MICROPROCESSORS AND EMBEDDED SYSTEM

PART A

1. Discuss the addressing modes of 8085 microprocessor with an example for each.

The process of identifying the operands for a particular instruction can be carried out in several ways. Addressing modes in 8085 is classified into 5 groups

Immediate addressing mode

Direct addressing mode

Register addressing mode

Register indirect addressing mode

Implicit addressing mode

Immediate addressing mode

In this type of addressing mode the operand is specified within the instruction itself.

Example

ADI 34H – This instruction adds the immediate data, 34H to the accumulator.

Direct Addressing Mode

In this mode of addressing, the address of the data (operand) is specified within the instruction.

Example:

OUT 10H

LDA 4100H

STA 2000H

Register Addressing Mode

In this mode, the data is copied from one register to another. **For example:** MOV A, B: means data in register B is copied to register A.

Example:

MOV A, B

Register Indirect Addressing Mode

In this mode the instruction specifies the name of the register in which the address of the data is available.

Example:

MOV A, M

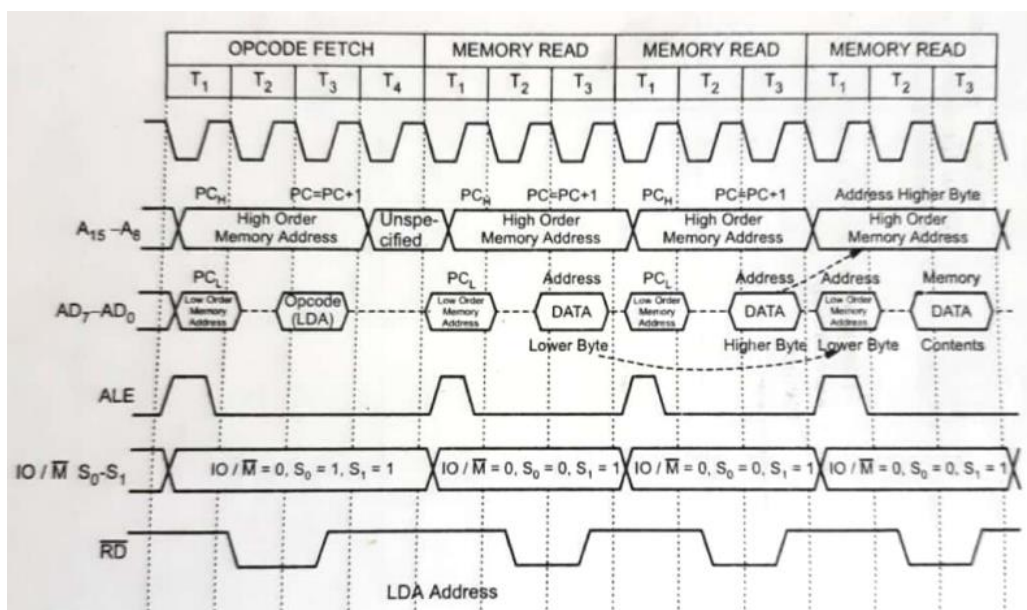
SUB M

DCR M

Implicit Addressing Mode

This mode doesn't require any operand; the data is specified by the opcode itself. **For example:** CMP.

2. Sketch the timing diagram of LDA 4500

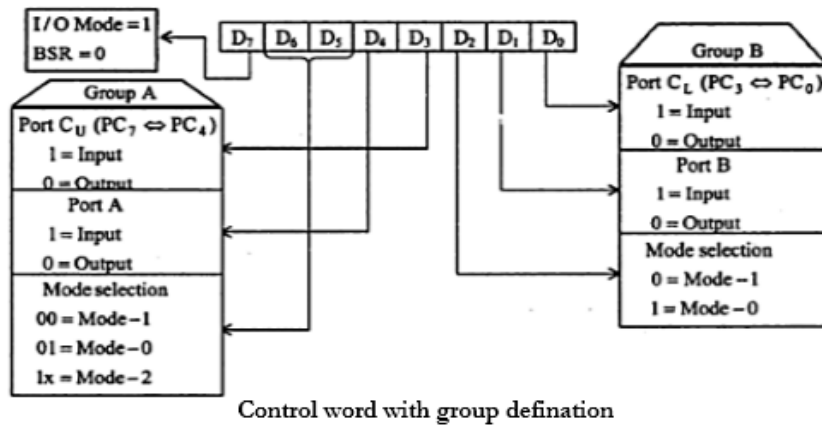


3. Explain the control word register of 8255

A high on the RESET pin causes all 24 lines of the three 8 bit ports to be in the input mode.

All flipflops are cleared and the interrupts are reset. This condition is maintained even after the RESET goes low.

The ports of the 8255 can then be programmed for any other mode by writing a single control word into the control register, when required.



Bit D0 :

- Sets Port C_{Lower} as input or output port.
- To make Port C_{Lower} as input port this bit set to 1.
- To make Port C_{Lower} as output port this bit set to 0.

Bit D1:

- Sets Port B as input or output port.
- To make Port B as input port this bit set to 1.
- To make Port B as output port this bit set to 0.

Bit D2:

- This bit for mode selection for the port B.
- If this bit set to 0, the port B will operate in mode 0.
- If this bit set to 1, the port B will operate in mode 1.

Bit D3:

- Sets Port C_{Upper} as input or output port.
- To make Port C_{Upper} as input port this bit set to 1.
- To make Port C_{Upper} as output port this bit set to 0.

Bit D4:

- Sets Port A as input or output port.
- To make Port A as input port this bit set to 1.
- To make Port A as output port this bit set to 0.

Bits D5 and D6 :

- These two bits mainly used for determining the I/O mode of port A.

Therefore, these bits are defined for the various modes of port A as follows.

D6	D5	Mode of port A
0	0	Mode 0
0	1	Mode 1
1	0 or 1	Mode 2

Bit D7:

- **This bit specifies either I/O function or bit set/reset function (BSR mode). If this bit set to 1 then the 8255 will work in I/O mode. If this bit set to 0 then the 8255 will work in BSR mode.**
- **There are 16 combinations of control words for various configurations of the ports of 8255 for Mode 0 operations. These control words are shown in table.**

4. Compare hard and soft real time systems

Hard Real Time Systems:

- A Hard Real-Time System guarantees that critical tasks complete on time. This goal requires that all delays in the system be bounded from the retrieval of the stored data to the time that it takes the operating system to finish any request made of it.
- Hard real time system is purely deterministic and time constraint system for example users expected the output for the given input in 10sec then system should process the input data and give the output exactly by 10th second. Here in the above example 10 sec. is the deadline to complete process for given data. Hard real systems should complete the process and give the output by 10th second. It should not give the output by 11th second or by 9th second, exactly by 10th second it should give the output. In the hard real time system meeting the deadline is very important if deadline is not met the system performance will fail.

Soft Real Time System

- A Soft Real Time System where a critical real-time task gets priority over other tasks and retains that priority until it completes.
- In soft real time system, the meeting of deadline is not compulsory for every time for every task but process should get processed and give the result. Even the soft real time systems cannot miss the deadline for every task or process according to the priority it should meet the deadline or can miss the deadline. If system is missing the deadline for every time the performance of the system will be worse and cannot be used by the users. Best example for soft real time system is personal computer, audio and video systems, etc.

5. List any five logical instruction of 8051

1. **ANL A, Rn**

Ex. **ANL A, R2** - Logically ANDs A and R2 and store result in A

2. **ANL A, direct**

Ex. **ANL A, 20H**

Logically ANDs contents of A and memory location whose address is 20H and

stores result in A

3. ANL A, @Ri

Ex. ANL A,@R1 - Logically ANDs contents of A and memory location whose address is given by R2 and stores result in A

4. ANL A,#DATA

Ex. ANL A,#50H - Logically ANDs contents of A with 50H and stores result in A

5. ANL DIRECT,A

Eg. ANL 20H, A – Logically ANDs contents of A with the contents of memory location 20H and stores result at memory location 20H

6. Explain the multifunction of Port 3 of 8051

- Port 3 serves for (i) Serial data communication (ii) External memory Read and Write control signals
- Also the port pin P3.7 functions as read control and the port pin P3.6 functions as write control for data memory.

PORT 3 ALTERNATE FUNCTIONS :

P3 BIT	FUNCTION	PIN
P3.0	RXD	10
P3.1	TXD	11
P3.2	$\overline{\text{INT0}}$	12
P3.3	$\overline{\text{INT1}}$	13
P3.4	TO	14
P3.5	TI	15
P3.6	$\overline{\text{WR}}$	16
P3.7	$\overline{\text{RD}}$	17

7. Explain the data types and directives of 8051

Directives are used to direct the assembler to do a address manipulation or allocation.

Directives are the statements for the assembler that are used for memory specifications. They deal with the addresses only and directly change the address of the next instruction or location of the current symbol.

- Microcontroller 8051 has only one 8-bits data type and the size of each register is also 8 bits.
- The job of the programmer is to break down data larger than 8 bits [00 to FFH, or 0 to 255 in decimal] to be processed by the CPU.
- The data type used by the 8051 can be positive or negative.

DB (Define Byte)

- The DB directive is used to define the 8-bit data and the numbers can be in decimal, binary, hex or ASCII formats.
- For decimal, the "D" after the decimal number is optional, but using "B" [binary] and "H" [hexadecimal] is required. The assembler will convert the numbers in hex.
- To indicate ASCII, simply put it in quotation marks 'like this'. The assembler will assign the ASCII code for the numbers or characters automatically.
- The DB directive is the only directive that can be used to define ASCII strings larger than two characters. It should be used for all ASCII data definitions.

ORG 300H

DATA1: DB 30 ;Decimal number [1E in HEX]

DATA2: DB 01010101 Binary ;Binary [55 in HEX]

DATA3: DB 32H ; equal to 32 in Hex, ie., 50 in decimal

DATA4: DB "A" ; equal to ASCII A

DATA5: DB "259" ; ASCII Character

DATA6: DB "MY NAME IS AYESHA" ; ASCII Character

Assembler Directives

The following are some more widely used directives of the 8051

ORG [Origin]:

- The ORG directive is used to indicate the beginning of the address.
- The number that comes after ORG can be either in HEX or in decimal.
- If the number is not followed by 'H', it is decimal and the assembler will convert it into hex.
- For Example ORG 0100H
- Some assemblers use ".ORG" [notice the dot] instead of "ORG" for the origin directive. For that you need to check your assembler.

EQU [Equate]:

- EQU is used to define a constant without occupying a memory location.
- The EQU directive does not set aside storage for a data item but associates a constant value with a data label so that when the label appears in the program, its constant value will be substituted for the label.
- Assume that there is a constant used in many different places in the program, and the programmer wants to change its value throughout
- Here uses EQU for the counter constant and then the constant is used to load the R4 register.
- Here uses EQU for the counter constant and then the constant is used to load the R4 register.

```
COUNT EQU 22
```

```
....    ....
```

```
MOV R4,#COUNT
```

After executing the instruction "MOV R4,#COUNT", the register R4 will be loaded with the value 22 [# sign indicates it is a value, notice it].

END Directive:

- END directive pseudo code is very important. END indicates to the assembler the end of the source [asm] file.
- The END directive is the last line of an 8051 program. In assembly language programming anything after the END directive is ignored by the assembler.
- Some assembler uses ".END" [notice the dot] instead of "END"

8. What is simplex, half duplex and full duplex in serial communication

Simplex

- In simplex transmission mode, the communication between sender and receiver occurs in only one direction. The sender can only send the data, and the receiver can only receive the data. The receiver cannot reply to the sender.
- Simplex transmission can be thought of as a one-way road in which the traffic travels only in one direction—no vehicle coming from the opposite direction is allowed to drive through.
- To take a keyboard / monitor relationship as an example, the keyboard can only send the input to the monitor, and the monitor can only receive the input and display it on the screen. The monitor cannot reply, or send any feedback, to the keyboard.
- Half Duplex

- The communication between sender and receiver occurs in both directions in half duplex transmission, but only one at a time. The sender and receiver can both send and receive the information, but only one is allowed to send at any given time. Half duplex is still considered a one-way road, in which a vehicle traveling in the opposite direction of the traffic has to wait till the road is empty before it can pass through.
- For example, in walkie-talkies, the speakers at both ends can speak, but they have to speak one by one. They cannot speak simultaneously.

Full Duplex

- In full duplex transmission mode, the communication between sender and receiver can occur simultaneously. The sender and receiver can both transmit and receive at the same time. Full duplex transmission mode is like a two-way road, in which traffic can flow in both directions at the same time.
- For example, in a telephone conversation, two people communicate, and both are free to speak and listen at the same time.

9. Write a program to convert a packed BCD number stored in location 2200 to binary and store in location 2230

	LDA 2500	Get the data in Accumulator
	MOV E A	Move A to E
	ANI F0	Mask the lower nibble
	RLC	Rotate the upper to lower nibble
	RLC	and save in register B
	RLC	
	RLC	
	MOV B A	Move it from A to B
	XRA A	Clear the accumulator
	MOV C, 0A	Initialize C as 0A
LOOP	ADD B	Add A and B
	DCR C	Decrement C register
	JNZ LOOP	
	MOV B A	

```

MOV A, E
ANI OF      Mask the upper nibble
ADD B
STA 2501
HLT

```

10. Explain CALL and RETURN instructions

CALL

- It is unconditional CALL used to call a subroutine program. When this instruction is executed, the address of the next instruction in the program counter is pushed to the stack.
- The 16 – bit address (which is the address of the subroutine program) given in the instruction is loaded in the program counter. Now, the processor will start executing the instructions stored in this call address.
- Three byte instruction
- Immediate addressing
- Five machine cycles (Opcode fetch (6T), MR(3T), MR(3T), MW(3T), MW (3T) = 18T

RETURN

- RET – Return to the main program
- It is an unconditional return instruction.
- This instruction is placed at the end of the subroutine program, in order to return to the main program. When this instruction is executed, the top of the stack is popped to (loaded in) the program counter.
- 1 byte instruction
- Register indirect addressing
- Three machine cycles (Opcode fetch (4T), MR(3T), MR(3T)) = 10T

11. Describe the following 8085 instructions (i) DAD Rp (ii) CMP M

DAD Rp

Add register pair to H and L registers

Add register pair to H and L registers The 16-bit contents of the specified register pair are added to the contents of the HL register and the sum is stored in the HL register.

The contents of the source register pair are not altered. If the result is larger than 16 bits, the CY flag is set. No other flags are affected. Eg: DAD H

CMP M

Compare register or memory with accumulator

The contents of the operand (register or memory) are subtracted from the contents of the accumulator. But both contents are preserved. The result of the comparison is shown by setting the flags of the PSW as follows:

if (A) < (reg/mem): carry flag is set

if (A) = (reg/mem): zero flag is set

if (A) > (reg/mem): carry and zero flags are reset

Example: CMP B or CMP M

b) Write a delay subroutine using register pair

- A subroutine is group of instruction written separately from the main program to perform a function that occurs repeatedly in the main program
- When a main program calls a subroutine the program execution is transferred to the subroutine after the completion of the subroutine, the program execution returns to the main program.
- The microprocessor uses the stack to store the return address of the subroutine.
- The 8085 has two instructions for dealing with subroutines.
- The CALL instruction is used to redirect program execution to the subroutine.
- The RET instruction is used to return to the main program at the end of the subroutine.
- The CALL instruction saves the address of the instruction following it and then transfers the program control to the first instruction in the subroutine.
- When subroutine execution is completed the RET instruction reads the return address from the stack and transfers control back to the instruction following the CALL.

Main program	DELAY Subroutine
6000H LXI SP, 3000H	6500H DELAY : MVI C, FFH
6003H -	6502H BACK : DCR C
-	6503H JNZ BACK
-	6506H RET
6010H CALL DELAY(6500H)	
6013H -	

- Here the main program initializes stack pointer at 3000H memory location and executes instructions in sequence till the execution of CALL instruction.
- After execution of CALL instruction program control is transferred to the delay subroutine stored at memory address 6500H.
- Before transfer of control to the subroutine program, the address of the instruction 6013H which is after the CALL instruction, is stored in the stack.
- At the end of delay subroutine RET instruction is executed, which reads the return address (6013) from the stack and transfers the program control to the instruction which is after CALL instruction.

12. Explain in detail the different flags of 8085

D7	D6	D5	D4	D3	D2	D1	D0
S	Z		AC		P		CY

- These are the set of 5 flip-flops . The five flip flops are

Sign (S)

Zero (Z)

Auxiliary Carry (AC)

Parity (P)

Carry (C)

- S- Sign flag

This flag is used with signed numbers. If D7 of the result in accumulator is 1 then sign flag is set otherwise reset. As we know that a number on the D7 always decides the sign of the number. If D7 is 1 the number is negative and if D7 is 0 the number is positive. It is irrelevant for operations of unsigned numbers.

- Zero flag (Z)

If the result stored in an accumulator is zero then this flip flop is set otherwise it is reset.

- Auxiliary Carry(AC)

If any carry goes from D3 to D4 in the output then it is set otherwise it is reset.

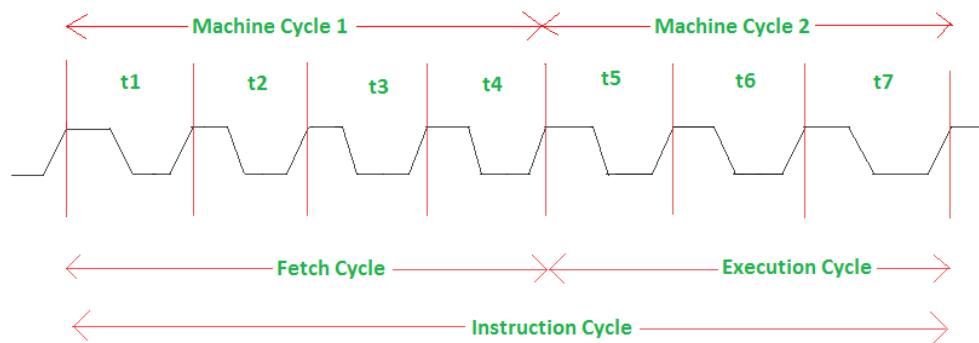
- Parity flag

If the no of 1's even in the output stored in the accumulator then it is set otherwise it is reset for the odd.

- Carry flag

If the result stored in an accumulator generates a carry from D7 in its final output then it is set otherwise it is reset.

b) Discuss instruction cycle, machine cycle and T state of 8085



Instruction cycle in 8085 microprocessor

- An instruction cycle can be defined as total time required by the microprocessor to completely fetch and execute an instruction
- An instruction cycle is comprised of one or more number of machine cycle
- An instruction cycle consists of a fetch cycle and execute cycle.
- In fetch cycle a CPU fetches Opcode(the machine code of an instruction) from the memory.
- The execute cycle is executed, to decode the instruction and to perform the work instructed by the instruction.
- The 8085 has seven machine cycles. These are
 1. Opcode Fetch
 2. Memory Read
 3. Memory Write
 4. I/O Read
 5. I/O Write
 6. Interrupt Acknowledge
 7. Bus Idle

T state

- One complete cycle of clock is called as T-state
- A subdivision of an operation (fetch, memory read, memory write) is carried out in one system clock period.
- In 8085, while a fetch cycle in all instructions requires 4T states, other machine cycle may require minimum 3 to a maximum of 6T states depending upon type of instruction.

13.(a) **Write a short note on different interrupts in 8085**

- The 8085 has multilevel interrupt system. It supports two Types of 8085 Interrupts:

Hardware

Software

- **Hardware** : Some pins on the 8085 allow, peripheral device to interrupt the main program for I/O operations.
- When an interrupt occurs, the 8085 completes the instruction it is currently executing and transfers the program control to a subroutine that services the peripheral device.
- Upon completion of the service routine, the CPU returns to the main program. These types of Types of 8085 Interrupts, where CPU pins are used to receive interrupt requests, are called **hardware interrupts**.
- **Software** : In software interrupts, the cause of the interrupt is an execution of the instruction. These are special instructions supported by the microprocessor.
- After execution of these instructions microprocessor completes the execution of the instruction it is currently executing and transfers the program control to the subroutine program.
- Upon completion of the execution of the subroutine program, program control returns to the main **program**.

13.(b) **List any five application domain of embedded system**

- The embedded systems have a huge variety of application domains. So, the embedded systems span all aspects of our modern life. The following are various applications of embedded systems.

1. **Home Appliances** Dishwasher, washing machine, microwave, Top-set box, security system, DVD, garden sprinkler systems etc..

2. **Office Automation** Fax, copy machine, smart phone system, modern, scanner, printers.

3. **Security** Face recognition, finger recognition, eye recognition, building security system, airport security system, alarm system.

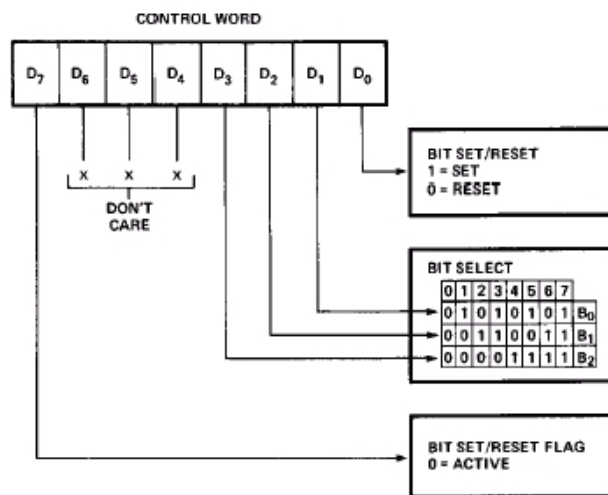
4. **Academia** Smart board, smart room, calculator, smart cord.

5. **Instrumentation** Signal generator, signal processor, power supplier, Process instrumentation,
6. **Telecommunication** Router, hub, cellular phone, IP phone, web camera

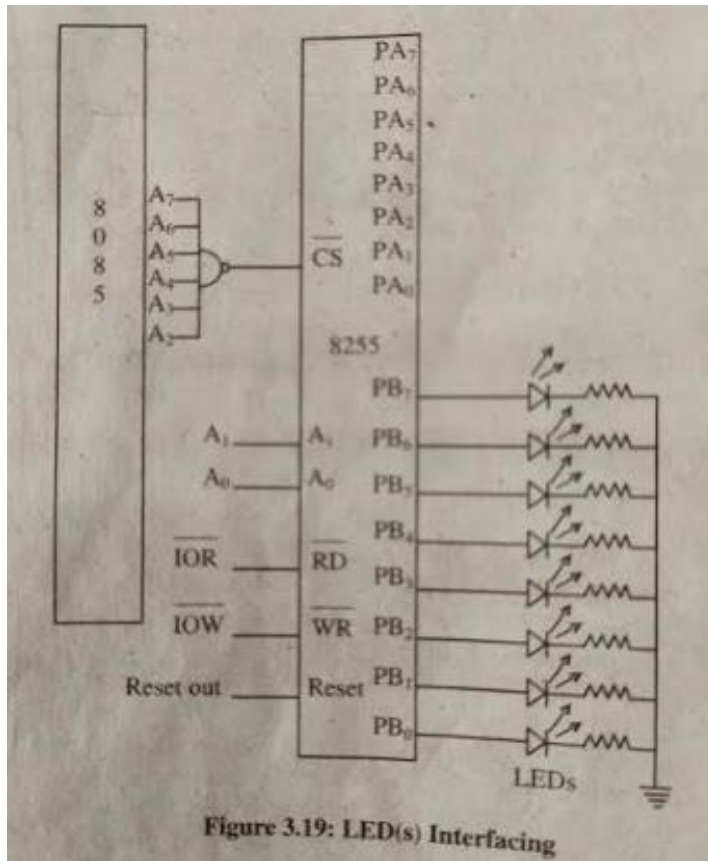
14 Explain the BSR operation of 8255 PPI

Bit set reset (BSR) mode

- The individual bits of Port C can be set or reset by sending out a single OUT instruction to the control register.
- When Port C is used for control/status operation, this feature can be used to set or reset individual bit.
- For the BSR mode, the MSB of the control word must be 0. The BSR mode handles only the bit set and resets operations for port C: PC0 to PC7 (both upper and lower).
- In this mode, the IC functions in such a way that it can assign any binary value to any bit of the Port C.



- 15 (a) Show the interfacing of LED with 8085 using 8255 PPI and write a program to continuously turn ON and OFF with a duty ratio of 0.5



- 8085 microprocessor is interfaced with input devices(DIP switches) and output devices (LED, LCD) using 8255 PPI.
- The figure shows the interfacing of DIP switches and LEDs with microprocessor.
- The Port B lines are connected to LEDs through protecting resistors.
- The other end is connected to ground.
- When logic 1 is send to any port B line, the LED will conduct current and it will glow.
- When logic 0 is send to port B then the LEDs will be off.

Program

MVI A, CW

OUT 43H

MVI A, 55H

OUT 41H

HLT

(b) **Differentiate between assembler and compiler**

Assembler	Compiler
Compiler translates high level programming language code to machine level code.	Assembler converts the assembly level language to machine level code.
Source code in high level programming language.	Assembly level code as input.
Compiler checks and converts the complete code at one time.	Assembler generally does not convert complete code at one time.
lexical analyzer, Syntax analyzer, Semantic analyzer, Code optimizer, Code generator, and Error handler	Assembler does works in two passes.
Mnemonic version of machine code.	Binary version of machine code.
C, C++ , Java compilers.	GAS, GNU assemblers.

15.(a) **Explain MOV, MOVX and MOVC**

MOV

Mov A, Rn

Example: Mov A, Ro

This instruction copies the contents of the register Ro of selected register bank to the accumulator.

2. Mov A, direct

Example: MOV A, 30H

This instruction copies the contents of memory location whose address is 30H to the accumulator.

MOVX

- The MOVX instruction transfer data between the Accumulator and a byte of external data memory, hence the “X” appended to MOV.

1. MOVX, A, @Ri

This instruction copies the contents of external memory whose address is given by register into the accumulator.

2. MOVX A, @DPTR

This instruction copies the contents of external memory whose address is given by DPTR register into the accumulator.

3. MOVX @Ri, A

This instruction copies the contents of accumulator into the external memory whose address is given by the register.

4. MOVX @DPTR, A

This instruction copies the contents of accumulator into the external memory whose address is given by the DPTR register.

MOVC

1. MOVC A, @A +DPTR

This instruction loads the accumulator from the contents of program memory whose address is given by the sum of the contents of accumulator and contents of DPTR register.

2. MOVC A, @A+PC

Copy the contents of the external ROM address formed by adding A and the PC, to A.

15.(b) Explain the various modes of operation of Timer of 8051

- In Intel 8051, there are two 16-bit timer registers. These registers are known as Timer0 and Timer1.
- The timer registers can be used in two modes. These modes are Timer mode and the Counter mode. The only difference between these two modes is the source for incrementing the timer registers.

Timer Mode

- In the timer mode, the internal machine cycles are counted. So this register is incremented in each machine cycle.
- So when the clock frequency is 12MHz, then the timer register is incremented in each millisecond. In this mode it ignores the external timer input pin.

Counter Mode

- In the counter mode, the external events are counted. In this mode, the timer register is incremented for each 1 to 0 transition of the external input pin.
- This type of transitions is treated as events. The external input pins are sampled once in each machine cycle, and to determine the 1 or 0 transitions, another machine cycle will be needed.

- So in this mode, at least two machine cycles are needed. When the frequency is 12MHz, then the maximum count frequency will be $12\text{MHz}/24 = 500\text{KHz}$. So for event counting the time duration is $2\ \mu\text{s}$.
- There are four different modes of the Timer or Counter. The Mode 0 to Mode 2 are for both of the Timer/Counter. Mode 3 has a different meaning for each timer register. There is a register called TMOD. This register can be programmed to configure these timers or counters.
- The Serial port is used for serial communication in mode 1 and 3. Timer1 is used for generating the baud rate. So only Timer0 is available for timer or counter operations.

16.(a) Explain the register banks of 8051

- The 8051 has 4K bytes of internal program memory (ROM) and 256 bytes of internal data memory (RAM).
- The Program Memory of the 8051 Microcontroller is used for storing the program to be executed i.e. instructions. The Data Memory on the other hand, is used for storing temporary variable data and intermediate results.

Data Memory (RAM) of 8051 Microcontroller

- The 8051's on-chip memory consists of 256 memory bytes organized as follows:

First 128 bytes:	00h to 1Fh Register Banks
	20h to 2Fh Bit Addressable RAM
	30 to 7Fh General Purpose RAM
Next 128 bytes:	80h to FFh Special Function Registers

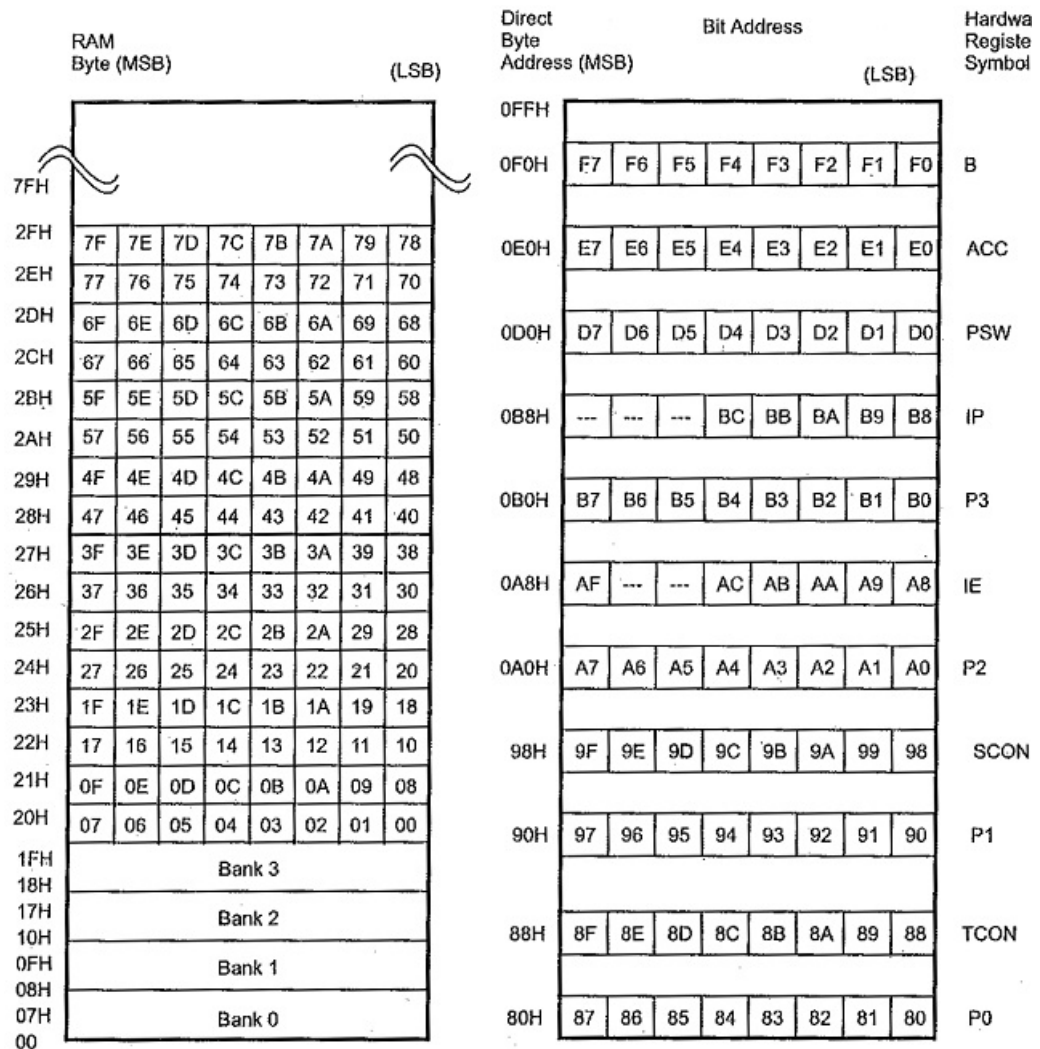


Fig. 12.12 (a) RAM bit address

Fig. 12.12 (b) SFR bit address

Internal RAM Organization

- It Consist of totally 256 RAM memory locations ; The lower 128 memory locations are called General Purpose Area. The internal RAM(this 128 memory locations) of 8051 is organized into three distinct areas.
 1. Register Bank
 2. Bit addressable
 3. General Purpose RAM

- A total of 32 bytes from location 00 to 1F hex are set aside for register banks and the stack.
- A total of 16 bytes from locations 20H to 2FH are set aside for bit – addressable read/write memory
- A total of 80 bytes from locations 30H to 7FH are used for read and write storage, or what is normally called a scratch pad. These 80 locations of RAM are widely used for the purpose of storing data and parameters by 8051 programmers
- The location 08H to 7FH can also be used as LIFO STACK. The upper 128 memory locations are called as SPECIAL PURPOSE AREA and consist of registers associated with ALU , on-chip MEMORIES and PERIPHERALS. The type of ON-CHIP RAM is SRAM [or Static RAM]

8051 register banks (working Registers)

- A total of 32 bytes of RAM are set aside for the register banks and stack.
- These 32 bytes are divide into 4 banks of registers in which each bank has 8 registers, R0 – R7.
- RAM locations from 0 to 7 are set aside for bank 0 of R0 - R7 where R0 is RAM location 0, R1 is RAM location 1, R2 is location 2, and so on, until memory location 7, which belongs to R7 of bank 0.
- The second bank of registers R0 to R7 starts at RAM location 08 and goes to location 0FH.
- The third bank of R0 – R7 starts at memory location 10H and goes to location 17H.
- Finally, RAM locations 18H to 1FH are set aside for allocated into 4 banks.

Bit – addressable RAM

- Of the 128 – byte RAM of the 8051, only 16 bytes are bit addressable. The rest must be accessed in byte format. The bit – addressable RAM locations are 20H to 2FH.
- These 16 bytes provide 128 bits of RAM bit - addressability since $16 \times 8 = 128$.
- They are addressed as 0 to 127 (in decimal) or 00 to 7FH. Therefore, the bit addresses 0 to 7 are for the first byte of internal RAM location 20H, and 8 to 0FH are the bit addresses of the second byte of RAM location 21H, and so on.
- The last byte of 2FH has bit addresses of 78H to 7FH. In the figure the internal RAM locations 20 – 2FH are both byte – addressable and bit addressable.

General Purpose RAM

- The RAM area above bit addressable area from 30H to 7FH is called general purpose RAM.

- It is addressable as byte.
- These 80 bytes of Internal RAM memory are available for general-purpose data storage.
- The only problem is that we have only 4 banks and very often the task of bank switching and keeping track of register bank usage is tedious and prone to errors,
- For this reason in many applications we use RAM locations 30-7FH as scratch pad and leave addresses 8 – 1FH for stack usage.
- That means that we use R0 – R7 of bank 0, and if we need more registers we simply use RAM locations 30 – 7FH.

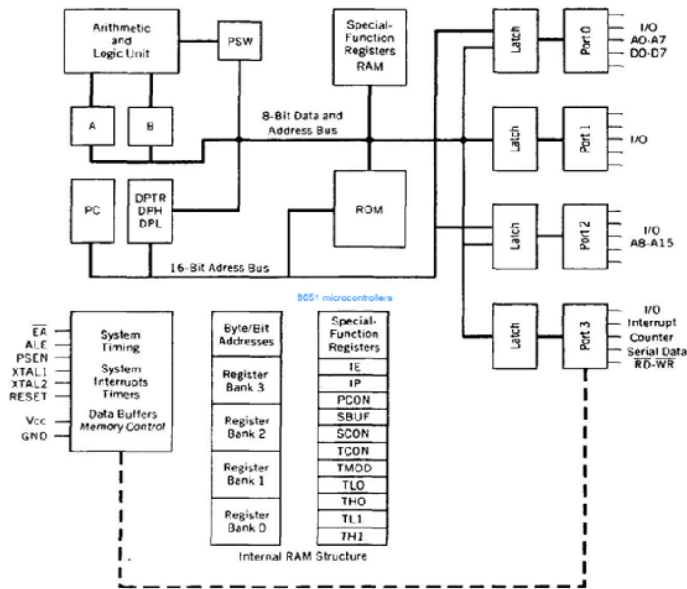
16. (b) Write an ALP in 8051 to multiply two numbers and store the result in external memory locations 4500 and 4501.

```

MOV R0, #12H    ; Get first no. in R0
MOV A, R0       ; Copy R0 value to accumulator
MOV R1, #05H    ; get second no in R1
MOV B, R1       ; Copy R1 value to B register
MUL AB          ; Multiply A by B
MOV 60H, A      ; Output stored in 60H data location
MOVX @4500, 60H
MOVX @4501, 61H

```

17(a) Draw the internal architecture of 8051 microcontroller



17. (b) Write a program for a counter to find the frequency of the signal connected to P3.4

```

MOV TMOD, #02H
MOV TH0, #0
AGAIN:  MOV R5, #250
        ACALL DELAY
        CPL P3.4
        SJMP AGAIN
DELAY:   SETB TR0
BACK:    JNB TF0, BACK
        CLR TR0
        CLR TF0 DJNZ R5, DELAY
        RET

```

$$T = 2 * (250 * 256 * 1.085)$$

$$f = 1/T = 7.2 \text{ Hz}$$